

UNIT-V – DYNAMIC PROGRAMMING

Dynamic Programming: General method, Applications- Matrix chain multiplication, Optimal binary search trees, 0/1 knapsack problem, All pairs shortest path problem, Travelling sales person problem, Reliability design.

-0-0-0-0-

INTRODUCTION

The drawback of greedy method is, we will make one decision at a time. This can be overcome in dynamic programming. In this we will make more than one decision at a time.

Dynamic programming is an algorithm design method that can be used when the solution to a problem can be viewed as the result of a sequence of decisions. Dynamic programming is applicable when the sub-problems are not independent, that is when sub-problems share sub-sub-problems. A dynamic programming algorithm solves every sub-sub-problem just once and then saves its answer in a table, thereby avoiding the work of re-computing the answer every time the sub-problem is encountered.

Definition:

It is a programming technique in which solution is obtained from a sequence of decisions

General Method:

The fundamental dynamic programming model may be written as,

$$F_n(R) = \max_{0 \leq R_n \leq R} \{P_n(R) + F_{n-1}(R - R_n)\}$$

Where $n = 2, 3, 4, \dots$

$$F_n(0) = 0$$

$$F_1(R) = P_1(R)$$

Once $F_1(R)$ is known equation(1) provides a relation for evaluation of $F_2(R)$, $F_3(R)$,.... This recursive process ultimately leads to the value of $F_{n-1}(R)$ and finally $F_n(R)$ at which process stops.

A dynamic programming problem can be divided into a number of stages where an optimal decision must be made at each stage. The decision made at each stage must take into account its effects not only on the next stage, but also on the entire subsequent stages. Dynamic programming provides a systematic procedure whereby starting with the last stage of the problem and working backwards one makes an optimal decision for each stage of problem. The information for the last stage is the information derived from the previous stage.

Dynamic programming design involves 4 major steps.

- 1) Characterize the structure of optimal solution.
- 2) Recursively define the value of an optimal solution.
- 3) Compute the value of an optimum solution in a bottom up fashion.
- 4) Construct an optimum solution from computed information.

The Dynamic programming technique was developed by Bellman based upon his principle known as principle of optimality. This principle states that “An optimal policy has the property that, what ever the initial decisions are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision”.

General Characteristics of Dynamic Programming:

The general characteristics of Dynamic programming are

- 1) The problem can be divided into stages with a policy decision required at each stage.
- 2) Each stage has number of states associated with it.
- 3) Given the current stage an optimal policy for the remaining stages is independent of the policy adopted.
- 4) The solution procedure begins by finding the optimal policy for each state of the last stage.
- 5) A recursive relation is available which identifies the optimal policy for each stage with n stages remaining given the optimal policy for each stage with $(n-1)$ stages remaining.

APPLICATIONS OF DYNAMIC PROGRAMMING

- 1) Matrix Chain Multiplication
- 2) Optimal Binary search Trees
- 3) 0/1 Knapsack Problem
- 4) Multi stage Graph
- 5) Traveling sales person problem
- 6) Reliability Design

1) MATRIX CHAIN MULTIPLICATION

Input: n matrices $A_1, A_2, A_3, \dots, A_n$ of dimensions $P_1 \times P_2, P_2 \times P_3, \dots, P_n \times P_{n+1}$ respectively.

Goal: To compute the matrix product $A_1 A_2 \dots A_n$.

Problem: In what order should $A_1 A_2 \dots A_n$ be multiplied so that it would take the minimum number of computations to derive the product.

Let A and B be two matrices of dimensions $p \times q$ and $q \times r$. Then $C = AB$. C is of dimension $p \times r$.

Thus C_{ij} takes q scalar multiplications and $q-1$ scalar additions.

Consider an example of the best way of multiplying 3 matrices.

Let A_1 of dimensions 5×4

A_2 of dimensions 4×6

A_3 of dimensions 6×2

$(A_1 A_2) A_3$ takes $(5 \times 4 \times 6) + (5 \times 6 \times 2) = 180$

$A_1 (A_2 A_3)$ takes $(5 \times 4 \times 2) + (4 \times 6 \times 2) = 88$

Thus $A_1 (A_2 A_3)$ is much cheaper to compute than $(A_1 A_2) A_3$, although both lead to the same final answer. Hence optimal cost is 88.

To solve this problem using dynamic programming method, we will perform the following steps.

Step 1: Let M_{ij} denote the cost of multiplying $A_i \dots A_j$ where the cost is measured in the number of multiplications.

Here, $M(i,i) = 0$ for all i and $M(1,n)$ is required solution.

Step 2: The sequence of decisions can be build using the principle of optimality. Consider the process of matrix chain multiplication.

Let T be the tree corresponding to the optimal way of multiplying $A_i \dots A_j$.

T has a left sub-tree L and right sub-tree R . L corresponds to multiplying $A_i \dots A_k$ and R to multiplying $A_{k+1} \dots A_j$ for some integer k such that $(i \leq k \leq j-1)$.

Thus we get optimal sub-chains of matrices and then the multiplication is performed. This ultimately proves that the matrix chain multiplication follows the principle of optimality.

Step 3: We will apply following formula for computing each sequence.

$$M_{ij} = \min \{ M_{ik} + M_{k+1,j} + P_i P_{k+1} P_{j+1} \mid i \leq k \leq j-1 \}$$

Example : Consider $A_1=5 \times 4$, $A_2=4 \times 6$, $A_3=6 \times 2$, $A_4=2 \times 7$.

$P_1=5$, $P_2=4$, $P_3=6$, $P_4=2$, $P_5=7$.

	1	2	3	4
0	$M_{11}=0$	$M_{22}=0$	$M_{33}=0$	$M_{44}=0$
1	$M_{12}=120$ $K=1$	$M_{23}=48$ $K=2$	$M_{34}=84$ $K=3$	
2	$M_{13}=88$ $K=1$	$M_{24}=104$ $K=3$		
3	$M_{14}=158$ $K=3$			

Computation of M_{ij}

The Computations are given as below.

$M_{ij} = \min\{M_{ik} + M_{k+1,j} + P_i P_{k+1} P_{j+1} \mid i \leq k \leq j-1\}$

Let $i=1$, $j=2$, $k=1$. We will compute M_{ij} .

$$M_{12} = M_{11} + M_{22} + P_1 P_2 P_3 = 0 + 0 + 5 \times 4 \times 6 = 120$$

Now $i=2$, $j=3$, $k=2$

$$M_{23} = M_{22} + M_{33} + P_2 P_3 P_4 = 0 + 0 + 4 \times 6 \times 2 = 48$$

Now $i=3$, $j=4$, $k=3$

$$M_{34} = M_{33} + M_{44} + P_3 P_4 P_5 = 0 + 0 + 6 \times 2 \times 7 = 84$$

Now $i=1$, $j=3$, $k=1$ or $k=2$. We will consider $k=1$ then

$$M_{13} = M_{11} + M_{23} + P_1 P_2 P_4 = 0 + 48 + 5 \times 4 \times 2 = 48 + 40 = 88$$

When $k=2$ we get M_{13} as,

$$M_{13} = M_{12} + M_{33} + P_1 P_3 P_4 = 120 + 0 + 60 = 120 + 60 = 180$$

As we get minimum value of M_{13} when $k=1$., we will set $k=1$ and $M_{13}=88$.

Now for $i=2$, $j=4$, $k=2$ or $k=3$ if when $k=2$

$$M_{24} = M_{22} + M_{34} + P_2 P_3 P_5 = 0 + 84 + 4 \times 6 \times 7 = 84 + 168 = 252$$

Now for $i=2$, $j=4$, when $k=3$

$$M_{24} = M_{23} + M_{44} + P_2 P_4 P_5 = 48 + 0 + 4 \times 2 \times 7 = 48 + 56 = 104$$

As we get minimum value of M_{24} when $k=3$, we will set $k=3$ and $M_{24}=104$.

Now for $i=1$, $j=4$, $k=1$ or 2 or 3 when $k=1$

$$M_{14} = M_{11} + M_{24} + P_1 P_2 P_5 = 0 + 104 + 5 \times 4 \times 7 = 104 + 140 = 244$$

Now for $i=1$, $j=4$, when $k=2$

$$M_{14} = M_{12} + M_{34} + P_1 P_3 P_5 = 120 + 84 + 5 \times 6 \times 7 = 120 + 84 + 210 = 414$$

Now for $i=1$, $j=4$, when $k=3$

$$M_{14} = M_{13} + M_{44} + P_1 P_4 P_5 = 88 + 0 + 5 \times 2 \times 7 = 88 + 70 = 158$$

As we get minimum value of M_{14} when $k=3$, we will set $k=3$ and $M_{14}=158$.

Thus we get optimum cost as 158. To seek the optimum sequence for this cost we will trace the algorithm Mul. For this cost 158 which is optimal.

The optimal sequence is $A_1 \times (A_2 \times A_3) \times A_4$.

For instance

$$A_1 = 5 \times 4$$

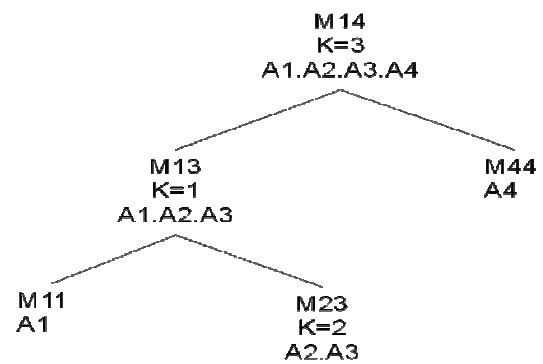
$$A_2 = 4 \times 6$$

$$A_3 = 6 \times 2$$

$$A_4 = 2 \times 7$$

$$\begin{aligned} \text{Hence } A_1 \times (A_2 \times A_3) \times A_4 \\ &= ((5 \times 4 \times 2) + (4 \times 6 \times 2)) + 2 \times 7 \times 5 \\ &= 40 + 48 + 70 \\ &= 158 \end{aligned}$$

Hence Optimal sequence is $A_1(A_2A_3)A_4$



Algorithm for Matrix Chain Multiplication

Algorithm matrix_chain_multiplication(array P[1..n])

```

{
  for i:= 1 to n do
    M[i,i]:=0;           // initialisation

  for len:=2 to n do
    {
      for i:=1 to (n - len +1) do
        {
          j:=i+len-1;
          M[i,j] := α ;
          for k:= i to j-1 do
            {
              // check all splits
              q:= M[i,k] + M[k+1,j]+p[i]*p[k+1]*p[j+1]
              if (q<M[i,j])
                {
                  M[i,j] := q;
                  S[i,j] := k;    //split matrix
                }
            }
          }
        }
      }
    }
  return M[1,n]; //Optimum value
}

```

Algorithm to compute optimum sequence

For extracting Optimum Sequence we can use the following algorithm.

```

Algorithm Mul(i,j)           //P=A[i]..A[k]
{                             //Q=A[k+1]..A[j]
  if (i==j) then
    Return A[i];
  else
    {
      K:= s[i,j];
      P:=Mul(i,k);
      Q:= Mul(k+1,j)
      return P*Q;
    }
}

```

In the algorithm , matrix_chain_mul first for loop takes $O(n)$ for initialization of $M[i,j]$. If $m[i,j]$ not computed previously then using three nested for loops each $m[i,j]$ called many times. This leads to $O(n^3)$ time complexity, of matrix_chain_mul algorithm.

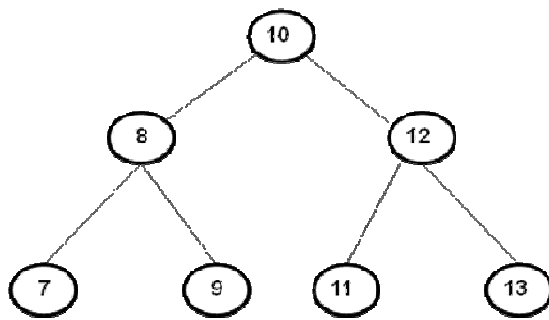
2) OPTIMAL BINARY SEARCH TREE

A Binary search tree is a binary tree that is either empty or in which every node contains a key and satisfies the following conditions.

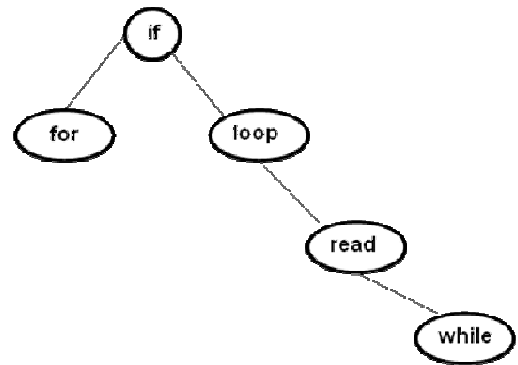
1. The key in the left child of a node (if it exists) is less than the key in its parent node.
2. The key in the right child of a node is greater than the key in its parent node.
3. The left and right subtrees of the root are again binary search trees.

No two entries in a binary search tree may have equal keys.

Ex) Binary Search Trees



(1)



(2)

If we want to search an element in binary search tree, first that element is compared with root node. If element is less than root node then search continue in left subtree. If element is greater than root node then search continue in right subtree. If element is equal to root node then print search was successful (element found) and terminate search procedure.

```
algorithm search(t,x)
{
    if (t==0) then return 0;
    else
        if (x = t->data) then return t;
        else
            if ( x < t->data) then return search (t->left, x);
            else
                return search (t->right, x);
}
```

The possible binary search trees for the identifier set (a1,a2,a3=do, if, stop)
Hence n=3

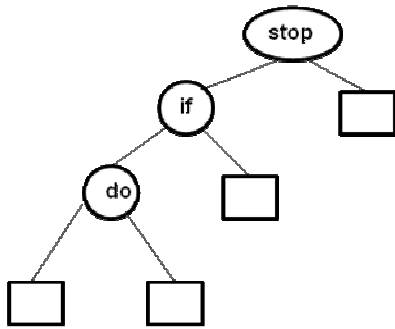
$$\begin{aligned} \text{The number of possible binary search trees} &= \frac{1}{n+1} {}^{2n}C_n = \frac{1}{3+1} {}^{2 \times 3}C_3 \\ &= \frac{1}{4} {}^6C_3 = \frac{1}{4} \times \frac{1 \times 2 \times 3 \times 4 \times 5 \times 6}{1 \times 2 \times 3 \times 1 \times 2 \times 3} = \frac{1}{4} \times 20 = 5 \end{aligned}$$

The sequence of key words to construct a BST may be one of the following

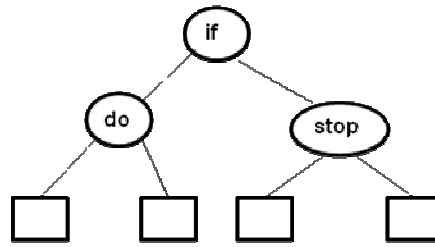
stop,if,do

if,do,stop

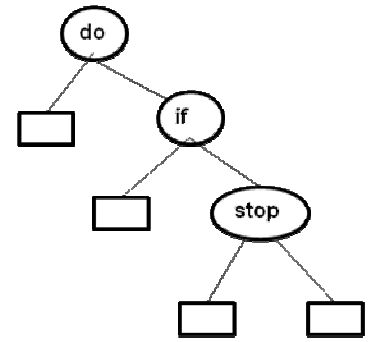
do,if,stop



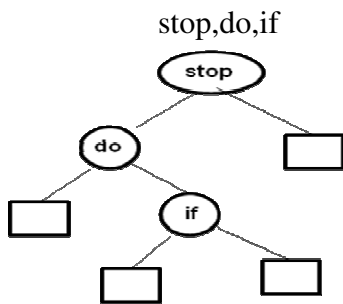
(a)



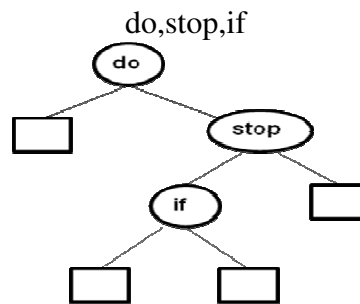
(b)



(c)



(d)



(e)

$$\text{cost}(T) = \sum_{1 \leq i \leq n} p(i) \times \text{level}(a_i) + \sum_{0 \leq i \leq n} q(i) \times \text{level}(E_i - 1)$$

$$p(i) = q(i) = \frac{1}{7}$$

$$a) 1 \times \frac{1}{7} + 2 \times \frac{1}{7} + 3 \times \frac{1}{7} + 1 \times \frac{1}{7} + 2 \times \frac{1}{7} + 3 \times \frac{1}{7} + 3 \times \frac{1}{7} = \frac{15}{7}$$

$$b) 1 \times \frac{1}{7} + 2 \times \frac{1}{7} + 2 \times \frac{1}{7} + 2 \times \frac{1}{7} + 2 \times \frac{1}{7} + 2 \times \frac{1}{7} + 2 \times \frac{1}{7} = \frac{13}{7}$$

$$c) 1 \times \frac{1}{7} + 2 \times \frac{1}{7} + 3 \times \frac{1}{7} + 1 \times \frac{1}{7} + 2 \times \frac{1}{7} + 3 \times \frac{1}{7} + 3 \times \frac{1}{7} = \frac{15}{7}$$

$$d) 1 \times \frac{1}{7} + 2 \times \frac{1}{7} + 3 \times \frac{1}{7} + 1 \times \frac{1}{7} + 2 \times \frac{1}{7} + 3 \times \frac{1}{7} + 3 \times \frac{1}{7} = \frac{15}{7}$$

$$e) 1 \times \frac{1}{7} + 2 \times \frac{1}{7} + 3 \times \frac{1}{7} + 1 \times \frac{1}{7} + 2 \times \frac{1}{7} + 3 \times \frac{1}{7} + 3 \times \frac{1}{7} = \frac{15}{7}$$

In the above binary search tree the cost of the tree 2 is minimum. Hence it is optimal binary search tree.

A binary search tree have maximum of two childs. i.e 0, 1 or 2 childs.

p(i) - probability of searching an internal node.

Q(i) – probability of searching an external node.

A successful search is terminated at an internal node denoted by circle (O) and unsuccessful search is terminated at an external node by square (□).

$$\text{cost}(T) = \sum_{1 \leq i \leq n} p(i) \times \text{level}(a_i) + \sum_{0 \leq i \leq n} q(i) \times \text{level}(E_i - 1)$$

Example 2) Let n=4 and (a1, a2, a3, a4) = (do, if, read, while) -- internal nodes

p(1:4)=(3,3,1,1)

q(0:4)=(2,3,1,1,1)

External nodes (E0, E1, E2, E3, E4)

As there are 4 internal nodes (a1, a2, a3, a4) = (do, if, read, while)

the number of possible binary search trees = $\frac{1}{n+1} {}^{2n}C_n = 14$

$w(i,j) = p(j) + q(j) + w(i,j-1)$ weight of t_{ij}

$c(i,j) = \min_{i < k \leq j} \{c(i,k-1) + c(k,j)\} + w(i,j)$ cost of t_{ij}

$r(i,j) = k$ root of t_{ij}

$W_{01} = E_0 a_1 E_1$

$W_{12} = E_1 a_2 E_2$

$W_{23} = E_2 a_3 E_3$

$W_{34} = E_3 a_4 E_4$

$W_{02} = E_0 a_1 E_1 a_2 E_2$

$W_{13} = E_1 a_2 E_2 a_3 E_3$

$W_{24} = E_2 a_3 E_3 a_4 E_4$

$W_{03} = E_0 a_1 E_1 a_2 E_2 a_3 E_3$

$W_{14} = E_1 a_2 E_2 a_3 E_3 a_4 E_4$

$W_{04} = E_0 a_1 E_1 a_2 E_2 a_3 E_3 a_4 E_4$

In order to solve above problem, first we will draw one table by taking 'I' corresponds to columns. The cells in the table can be indicated as $w_{j,j+i}, c_{j,j+i}, R_{j,j+i}$.

	0	1	2	3	4
0	W ₀₀ =2 C ₀₀ =0 R ₀₀ =0	W ₁₁ =3 C ₁₁ =0 R ₁₁ =0	W ₂₂ =1 C ₂₂ =0 R ₂₂ =0	W ₃₃ =1 C ₃₃ =0 R ₃₃ =0	W ₄₄ =1 C ₄₄ =0 R ₄₄ =0
1	W ₀₁ = C ₀₁ = R ₀₁ =	W ₁₂ = C ₁₂ = R ₁₂ =	W ₂₃ = C ₂₃ = R ₂₃ =	W ₃₄ = C ₃₄ = R ₃₄ =	
2	W ₀₂ = C ₀₂ = R ₀₂ =	W ₁₃ = C ₁₃ = R ₁₃ =	W ₂₄ = C ₂₄ = R ₂₄ =		
3	W ₀₃ = C ₀₃ = R ₀₃ =	W ₁₄ = C ₁₄ = R ₁₄ =			
4	W ₀₄ = C ₀₄ = R ₀₄ =				

Principle of optimality was applied.

The first row is initiated as $W(i,i)=q(i)$ $R(i,i)=0$ $C(i,i) = 0$

It means

$$W_{00} = q(0) = 2 \quad C_{00} = R_{00} = 0$$

$$W_{11} = q(1) = 3 \quad C_{11} = R_{11} = 0$$

$$W_{22} = q(2) = 1 \quad C_{22} = R_{22} = 0$$

$$W_{33} = q(3) = 1 \quad C_{33} = R_{33} = 0$$

$$W_{44} = q(4) = 1 \quad C_{44} = R_{44} = 0$$

The remaining values of cells can be calculated using equations as follows

$$w(i,j)=p(j) + q(j) + w(i,j-1)$$

$$w(0,1)=p(1) + q(1) + w(0,0) = 3+3+2 = 8$$

$$w(1,2) = p(2) + q(2) + w(1,1) = 3+1+3 = 7$$

$$w(2,3) = p(3) + q(3) + w(2,2) = 1+1+1 = 3$$

$$w(3,4) = p(4) + q(4) + w(3,3) = 1+1+1 = 3$$

$$w(0,2) = p(2) + q(2) + w(0,1) = 3+1+8 = 12$$

$$w(1,3) = p(3) + q(3) + w(1,2) = 1+1+7 = 9$$

$$w(2,4) = p(4) + q(4) + w(2,3) = 1+1+3 = 5$$

$$w(0,3) = p(3) + q(3) + w(0,2) = 1+1+12 = 14$$

$$w(1,4) = p(4) + q(4) + w(1,3) = 1+1+9 = 11$$

$$w(0,4) = p(4) + q(4) + w(0,3) = 1+1+14 = 16$$

$$c(i, j) = \min_{i < k \leq j} \{c(i, k-1) + c(k, j)\} + w(i, j)$$

$$C(0,1) = \min \{ c(0,0) + c(1,1) \} + w(0,1) = 0 + 0 + 8 = 8 \quad \text{when } k=1$$

$$R(0,1)=1$$

$$C(1,2) = \min \{ c(1,1) + c(2,2) \} + w(1,2) = 0 + 0 + 7 = 7 \quad \text{when } k=2$$

$$R(1,2)=2$$

$$C(2,3) = \min \{ c(2,2) + c(3,3) \} + w(2,3) = 0 + 0 + 3 = 3 \quad \text{when } k=3$$

$$R(2,3)=3$$

$$C(3,4) = \min \{ c(3,3) + c(4,4) \} + w(3,4) = 0 + 0 + 3 = 3 \quad \text{when } k=4$$

$$R(3,4)=4$$

When $k=2$

$$C(0,2) = \min \{ c(0,1) + c(2,2) \} + w(0,2) = 8 + 0 + 12 = 20$$

When $k=1$

$$C(0,2) = \min \{ c(0,0) + c(1,2) \} + w(0,2) = 0 + 7 + 12 = 19$$

$$C(0,2) = \min\{20,19\} = 19$$

$$r(0, 2) = 1$$

When $k=3$ or 2

$$C(1,3) = \min \{ c(1,1) + c(2,3), c(1,2) + c(3,3) \} + w(1,3) = \{ 0 + 3, 7 + 0 \} + 9 = 12$$

$$r(1, 3) = 2$$

When $k=3$

$$C(2,4) = \min \{ c(2,2) + c(3,4) \} + w(2,4) = 0 + 3 + 5 = 8$$

When $k=4$

$$C(2,4) = \min \{ c(2,3) + c(4,4) \} + w(2,4) = 3 + 0 + 5 = 8$$

$$r(2,4)=3$$

When $k=1$ or 2 or 3

$$C(0,3) = \min \{ c(0,0) + c(1,3), c(0,1) + c(2,3), c(0,2) + c(3,3) \} + w(1,3) \\ = \min\{12, 11, 10\} + 14 = 10 + 14 = 24$$

$$r(0,3)=2$$

When $k=2$ or 3 or 4

$$C(1,4) = \min \{ c(1,1) + c(2,4), c(1,2) + c(3,4), c(1,3) + c(4,4) \} + w(1,4) \\ = \min\{0+8, 7+3, 12+0\} + 11 = 8 + 11 = 19$$

$$r(1,4)=2$$

When $k=1$ or 2 or 3 or 4

$$C(1,4) = \min \{ c(0,0) + c(1,4), c(0,1) + c(2,4), c(0,2) + c(3,4), c(0,3) + c(4,4) \} + w(0,4) \\ = \min\{19, 16, 23, 25\} + 16 = 16 + 16 = 32$$

$$R(0,4)=2$$

	0	1	2	3	4
0	W ₀₀ =2 C ₀₀ =0 R ₀₀ =0	W ₁₁ =3 C ₁₁ =0 R ₁₁ =0	W ₂₂ =1 C ₂₂ =0 R ₂₂ =0	W ₃₃ =1 C ₃₃ =0 R ₃₃ =0	W ₄₄ =1 C ₄₄ =0 R ₄₄ =0
1	W ₀₁ =8 C ₀₁ =8 R ₀₁ =1	W ₁₂ =7 C ₁₂ =7 R ₁₂ =2	W ₂₃ =3 C ₂₃ =3 R ₂₃ =3	W ₃₄ =3 C ₃₄ =3 R ₃₄ =4	
2	W ₀₂ =12 C ₀₂ =19 R ₀₂ =1	W ₁₃ =9 C ₁₃ =12 R ₁₃ =2	W ₂₄ =5 C ₂₄ =8 R ₂₄ =3		
3	W ₀₃ =14 C ₀₃ =25 R ₀₃ =2	W ₁₄ =11 C ₁₄ =19 R ₁₄ =2			
4	W ₀₄ =16 C ₀₄ =32 R ₀₄ =2				

Now observe the table's last cell i.e. 4th row 0th column, contains $r_{04} = 2$ i.e. $r_{04} = a_2$ (2 corresponds to second node a_2).

$R_{04}=k$, then $k=2$

To build OBST $R(0,4)=2=k=2$

Hence a_2 becomes the root node

Let T be OBST $T_{i,j} = T_{i,k-1}$, and $T_{k,j}$

T_{04} is divided into two parts i.e. T_{01} and T_{24}

$$T_{01}=r(0,1)=1=k=1$$

$$T_{24}=r(2,4)=3=k=3$$

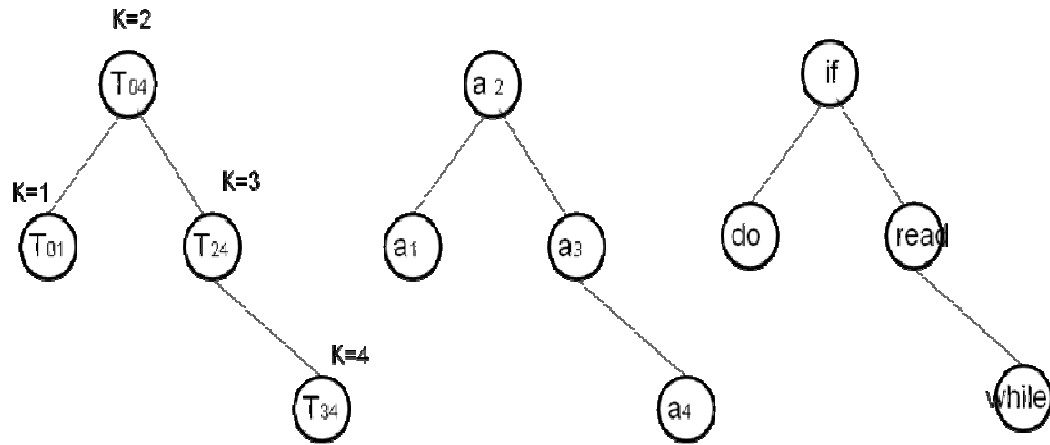
T_{01} is divided into two parts T_{00} and T_{11} where $k=1$

T_{24} is divided into two parts T_{22} and T_{34} where $k=3$

T_{34} is divided into two parts T_{33} and T_{44} where $k=4$

Since $r_{00}, r_{11}, r_{22}, r_{33}, r_{44} = 0$ these are external nodes and can be neglected.

Let T be the optimal binary search tree



Algorithm OBST(p,q,n)

```

{
  for i:=0 to n-1 do
  {
    w[i,i]:=q[i];
    r[i,i]:=0;
    c[i,i]:=0;
    w[i,i+1]:=q[i]+q[i+1]+p[i+1];
    r[i,i+1]:=i+1;
    c[i,i+1]=q[i]+q[i+1]+p[i+1];
  }
  w[n,n]:=q[n];
  r[n,n]:=0;
  c[n,n]:=0.0;
  for m:=2 to n do
    for i:=0 to n-m do
    {
      j:=i+m;
      w[i,j]:=w[i,j-1]+p[j]+q[j];
      k:=find(c, r, i, j);
      c[i,j]:=w[i,j]+c[i,k-1]+c[k,j];
      r[i,j]:=k;
    }
  write(c[0,n],w[0,n],r[0,n]);
}

```

algorithm find(c, r, i, j)

```

{
  min := ∞;
  for m := r[i, j-1] to r[i+1, j] do
    if ((c[i, m-1]+c[m, j]) < min) then
    {
      min:=c[i, m-1]+c[m, j];
      l:=m;
    }
  return l;
}

```

Time complexity $O(n \log(n))$

3) 0/1 KNAPSACK PROBLEM

In this problem, 'n' objects are given. with each object 'i' having a weight of 'w_i' & a knapsack having a capacity of 'm' is given. If an object 'i' is placed in the knapsack, a profit of 'P_ix_i' is earned. A solution to this knapsack problem can be obtained by making a sequence of decisions on the variables x₁, x₂, x₃,... x_n. A decision of variable x_i involves in determining which of the values 0 or 1 is to be assigned to it.

Following a decision on any object x_i. We may be in any of 2 possible states.

- 1) The capacity remaining in the knapsack is m and no profit has earned. (object=0)
- 2) The capacity remaining is m-w_i and a profit of p_i has earned.

It is clear that the remaining decisions x_{i+1}, x_{i+2},...x_n must be optimal w.r.t. the problem state resulting from the decision of x_i (i=1). Hence the principle of optimality holds.

Suppose that a store contains different types of ornaments, which are made up of gold.

Let n₁, n₂, n₃ be ornaments, cost and weight of these ornaments are c₁, c₂, c₃ dollars w₁, w₂, w₃ pounds respectively. Now a thief wants to rob the ornaments such that he should get maximum profit. In this the thief can't place fraction of ornament in the bag, i.e. either he can place ornament completely in the bag or he can't place ornament. So

X_i= 0 or 1.

X_i = 0 means we can not place ornament in the bag.

X_i = 1 means we can place ornament completely in the bag.

This problem contains either 0 or 1, hence the problem is called 0/1 Knapsack problem.

Example : Consider the knapsack instance n=3, (w₁, w₂, w₃) = (2, 3, 4), (p₁, p₂, p₃) = (1, 2, 5) and m=6

$$(p_1, w_1) = (1, 2)$$

$$(p_2, w_2) = (2, 3)$$

$$(p_3, w_3) = (5, 4)$$

$$s^0 = \{0, 0\}$$

$$s_1^i = s^{i-1} + (p_i, w_i)$$

$$s_1^1 = s^0 + (p_1, w_1) \quad \text{-----addition}$$

$$= \{(0, 0)\} + \{(1, 2)\}$$

$$s_1^1 = \{(1, 2)\}$$

$$s^1 = s^0 + s_1^1 \quad \text{-----merging } s^0 \text{ and } s_1^1 \text{ results } s^1$$

$$s^1 = \{(0, 0)\} + \{(1, 2)\} = \{(0, 0), (1, 2)\}$$

$$s_1^2 = s^1 + (p_2, w_2) \quad \text{-----addition}$$

$$= \{(0, 0), (1, 2)\} + \{(2, 3)\}$$

$$= \{(2, 3), (3, 5)\}$$

$$s^2 = s^1 + s_1^2 \quad \text{-----merging}$$

$$s^2 = \{(0, 0), (1, 2)\} + \{(2, 3), (3, 5)\}$$

$$s^2 = \{(0, 0), (1, 2), (2, 3), (3, 5)\};$$

$$s_1^3 = s^2 + (p_3, w_3) \text{ -----addition}$$

$$= \{(0,0), (1,2), (2,3), (3,5)\} + \{5,4\}$$

$$= \{(5,4), (6,6), (7,7), (8,9)\}$$

$$s^3 = s^2 + s_1^3 \text{ -----merging}$$

$$s^3 = \{(0,0), (1,2), (2,3), (3,5)\} + \{(5,4), (6,6), (7,7), (8,9)\}$$

$$s^3 = \{(0,0), (1,2), (2,3), (3,5), (5,4), (6,6), (7,7), (8,9)\};$$

Using purge rule(dominance rule) in the set S^3 on ordered pairs (3,5) (5,4) i.e. $3 < 5$ and $5 > 4$. so we can eliminate (3,5) from S^3 .

As knapsack maximum capacity is 6 we can eliminate (7,7),(8,9) from S^3 .
Now $S^3 = \{(0,0), (1,2), (2,3), (5,4), (6,6)\}$.

After applying purge rule we will check the following condition inorder to find solution

If $(p_i, w_i) \in s^n$ and $(p_i, w_i) \notin s^{n-1}$ then $x_n = 1$ otherwise $x_n = 0$

$$(6,6) \in s^3 \text{ and } (6,6) \notin s^2 \text{ so } x_3 = 1 \quad (6,6) - (5,4) = (1,2)$$

$$(1,2) \in s^2 \quad (1,2) \notin s^1 \rightarrow \text{False} \quad x_2 = 0$$

$$(1,2) \in s^1 \quad (1,2) \notin s^0 \rightarrow \text{True} \quad x_1 = 1$$

$$\therefore x_1 = 1, x_2 = 0, x_3 = 1$$

$$\text{Maximum profit is } \sum p_i x_i = p_1 x_1 + p_2 x_2 + p_3 x_3 = 1 \times 1 + 2 \times 0 + 5 \times 1 = 1 + 5 = 6$$

$$\text{Optimal solution is } (x_1, x_2, x_3) = (1, 0, 1)$$

$$\text{Max Profit is } \sum p_i x_i = 6$$

4) MULTISTAGE GRAPH: A multi stage Graph $G=(V,E)$ is a directed graph in which the vertices are partitioned into $k \geq 2$ disjoint sets, V_i where $1 \leq i \leq k$. If $\langle u,v \rangle$ is an edge in E , then $u \in V_i, v \in V_{i+1}$ where $1 \leq i \leq k$. The sets V_i and V_k are such that $|V_i| = |V_k| = 1$. Let s and T be 2 vertices in V_i and V_k respectively then the vertex s is called the source and T is called the Sink” or “Destination”. Let $c(i,j)$ be the cost of the edge $\langle i,j \rangle$. The cost of a path from S to T is the sum of the costs of edges on the path. The multistage graph problem is to find minimum cost path from S to T . Each set V_i defines a stage in the graph. Because of the constraints on E , every path from s to t starts in stage 1, goes to stage2, then stage 3, then to stage 4 and so on. And eventually terminates in stage k . figure shows a five stage graph. A minimum cost s to t path is indicated by broken edges.

Consider a resource allocation problem in which n units of resources are to be allocated to r projects. If j $0 \leq j \leq n$, units of the resource are allocated to project I , then the resulting net profit is $N(i,j)$. The problem is to allocate the resource to the r projects is such a way as to maximize total net profit. This problem can be formulated as $r+1$ stage graph problem.

A dynamic programming formulation for a k -stage graph problem is obtained by first noticing that every S to T path is the result of $k-2$ decisions. The i^{th} decision involves in determining which vertex v_{i+1} where $1 \leq i \leq k-2$ is to be on the path. Let $p(i,j)$ be a minimum cost path from vertex ‘ j ’ in V_i to vertex ‘ T ’. Let $cost(i,j)$ be the cost of this path. This multistage graph problem can be solved using 2 approaches.

- 1) Forward approach
- 2) Backward approach

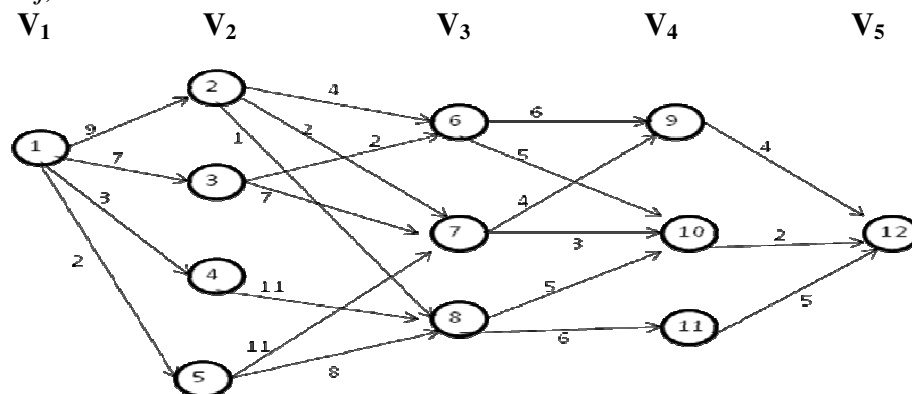
Forward Approach:

$$cost(i,j) = \min \{ c(j,l) + cost(i+1,l) \}$$

$$l \in V_{i+1}$$

$$\langle j,l \rangle \in E$$

STAGES



Five-Stage Graph

First compute $cost[k-2, j] \forall j \in V_{k-2}$ and then compute $cost[k-3, j] \forall j \in V_{k-3}$ and so on, and finally compute $cost[1, S]$

There are 5 stages in the graph $V_1 V_2 V_3 V_4 V_5$

Stage 4 contains 3 vertices (9,10,11) cost of each vertex at stage 4 to the destination

$cost(4,9) = c(9,12) = 4$ stage 4 cost from vertex 9 to vertex 12

$cost(4,10) = c(10,12) = 2$ stage 4 cost from vertex 10 to vertex 12

$cost(4,11) = c(11,12) = 5$ stage 4 cost from vertex 11 to vertex 12

Stage 1: $\text{cost}[k-2, j] \forall j \in v_{k-2}$

$$\text{cost}[3, j] \forall j \in v_3 = \{6, 7, 8\}$$

$\text{cost}(3,6)$ means that 3 refers to stage. At stage 3 cost for vertex 6 to destination

$$\text{cost}(3,6) = \min \begin{cases} c(6,9) + \text{cost}(4,9) = 6 + 4 = 10 \\ c(6,10) + \text{cost}(4,10) = 5 + 2 = 7 \end{cases} \quad 7$$

$$\text{cost}(3,7) = \min \begin{cases} c(7,9) + \text{cost}(4,9) = 4 + 4 = 8 \\ c(7,10) + \text{cost}(4,10) = 3 + 2 = 5 \end{cases} \quad 5$$

$$\text{cost}(3,8) = \min \begin{cases} c(8,10) + \text{cost}(4,10) = 5 + 2 = 7 \\ c(8,11) + \text{cost}(4,11) = 6 + 5 = 11 \end{cases} \quad 7$$

$$\text{cost}(2,2) = \min \begin{cases} 4 + \text{cost}(3,6) \\ 2 + \text{cost}(3,7) \\ 1 + \text{cost}(3,8) \end{cases} \quad 7$$

$$\text{cost}(2,3) = 9$$

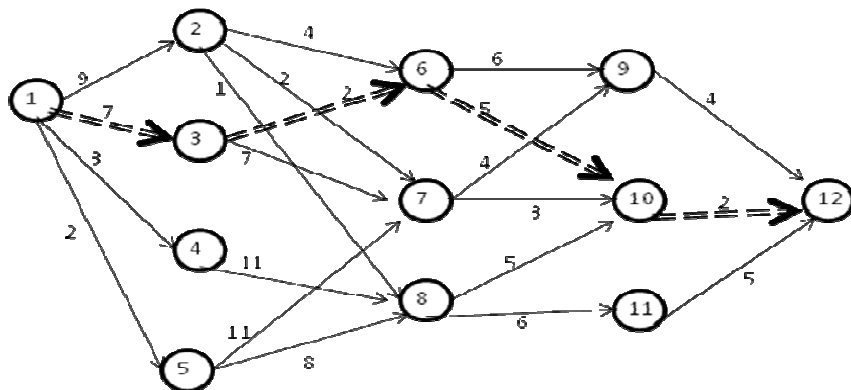
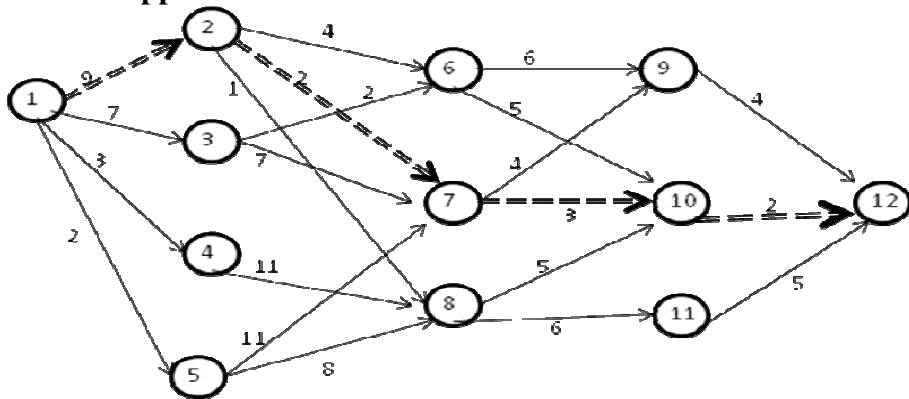
$$\text{cost}(2,4) = 18$$

$$\text{cost}(2,5) = 15$$

$$\text{cost}(1,1) = \min \begin{cases} 9 + \text{cost}(2,2) \\ 7 + \text{cost}(2,3) \\ 3 + \text{cost}(2,4) \\ 2 + \text{cost}(2,5) \end{cases} \quad 16$$

Note that in the calculation of $\text{cost}(2,2)$, we have reused the values of $\text{cost}(3,6)$, $\text{cost}(3,7)$ and $\text{cost}(3,8)$ and so avoided their recomputation. A minimum cost s to t path has a cost of 16. This path can be determined easily if we record the decision made at each stage (vertex).

Backward approach:



5) TRAVELLING SALES PERSON

The tour of graph G is a directed simple cycle that includes every vertex in V . The cost of the tour is the sum of cost of the edges on the tour. The travelling salesperson problem is to find a tour of minimum cost.

Applications: Suppose we have to route a postal van to pick up mail from mailbox located at 'n' different sites. An $(n+1)$ vertex graph can be used to represent the situation. One vertex represents the post office from which the postal van starts and to which it must return. Edge $\langle i, j \rangle$ is assigned a cost equal to the distance from site 'I' to site 'j'. Route taken by the postal van is a tour and we are interested in finding tour of minimum length.

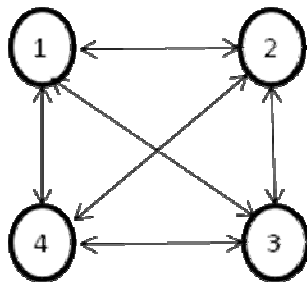
Every tour consist of an edge $\langle 1, k \rangle$ for some $k \in V - \{1\}$ and a path from vertex k to vertex 1, which goes through each vertex in $V - \{1, k\}$ exactly once. It is easy to see that if the tour is optimal, then the path from k to 1 must be a shortest k to 1 path going through all vertices in $V - \{1, k\}$. Hence, the principle of optimality holds.

Let $g(i, S)$ be the length of a shortest path starting at vertex 'i' going through all vertices in 'S' & terminating at vertex 1. The function $g(1, V - \{1\})$ is the length of an optimal salesperson tour. From the principle of optimality

$$g(1, V - \{1\}) = \min_{2 \leq k \leq n} \{ C_{1k} + g(k, V - \{1, k\}) \}$$

Generalizing the above equation we obtain (for $i \notin S$)

$$g(i, S) = \min_{j \in S} \{ C_{ij} + g(j, S - \{j\}) \}$$



	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Find $g(1, \{2, 3, 4\})$

$V \setminus \{1, 2, 3, 4\}$

Solution:

Stage 1) compute $g(i, S)$ where $|S| = \emptyset$, **with no intermediate nodes**.

$$i \in \{v - \{1\}\} \quad i \neq 1, 1 \notin S \text{ and } i \notin S$$

$$i = \{2, 3, 4\}$$

$$g(2, \emptyset) = c_{21} = 5$$

$$g(3, \emptyset) = c_{31} = 6$$

$$g(4, \emptyset) = c_{41} = 8$$

Stage 2) Compute $g(i, S)$ where $|S| = 1$, **with one intermediate node**

$$i \in \{v - \{1\}\} \quad i \neq 1, 1 \notin S \text{ and } i \notin S$$

$$i = \{2, 3, 4\} \quad s = \{2/3/4\} \text{ \& } \langle i, s \rangle \text{ should be an edge } (j=s)$$

$$g(i, S) = \min_{j \in S} \{ C_{ij} + g(j, S - \{j\}) \}$$

$$g(2, \{3\}) = c_{23} + g(3, S - \{3\}) = 9 + g(3, \emptyset) = 9 + c_{31} = 9 + 6 = 15$$

$$g(2, \{4\}) = c_{24} + g(4, S - \{4\}) = 10 + g(4, \emptyset) = 10 + c_{41} = 10 + 8 = 18$$

$$g(3, \{2\}) = c_{32} + g(2, S - \{2\}) = 13 + g(2, \emptyset) = 13 + c_{21} = 13 + 5 = 18$$

$$g(3, \{4\}) = c_{34} + g(4, S - \{4\}) = 12 + g(4, \emptyset) = 12 + c_{41} = 12 + 8 = 20$$

$$g(4, \{2\}) = c_{42} + g(2, S - \{2\}) = 8 + g(2, \emptyset) = 8 + c_{21} = 8 + 5 = 13$$

$$g(4, \{3\}) = c_{43} + g(3, S - \{3\}) = 9 + g(3, \emptyset) = 9 + c_{31} = 9 + 6 = 15$$

Step 3) Compute $g(i,s)$ where $|S|=2$, **with two intermediate nodes**

$$i \in (V - \{2\}) \quad i \neq 1, 1 \notin S \text{ and } i \notin S$$

$$i = \{2, 3, 4\}, \quad S = \{\{2, 3\} / \{3, 4\} / \{2, 4\}\}$$

$$g(i, S) = \min_{j \in S} \{C_{ij} + g(j, S - \{j\})\}$$

$$g(2, \{3, 4\}) = \min \begin{cases} c_{23} + g(3, S - \{3\}) & j = \{3, 4\} \\ c_{24} + g(4, S - \{4\}) & \text{when } j = 3 \\ & \text{when } j = 4 \end{cases}$$

$$\min \begin{cases} c_{23} + g(3, \{4\}) \\ c_{24} + g(4, \{3\}) \end{cases} = \min \begin{cases} 9 + 20 \\ 10 + 15 \end{cases} = \min\{29, 25\} = 25$$

$$g(3, \{2, 4\}) = \min \begin{cases} c_{32} + g(2, S - \{2\}) \\ c_{34} + g(4, S - \{4\}) \end{cases} \quad j = \{2, 4\}$$

$$\min \begin{cases} 13 + g(2, \{4\}) \\ 12 + g(4, \{2\}) \end{cases} = \min \begin{cases} 13 + 18 \\ 12 + 13 \end{cases} = 25$$

$$g(4, \{2, 3\}) = \min \begin{cases} c_{42} + g(2, S - \{2\}) \\ c_{43} + g(3, S - \{3\}) \end{cases} \quad j = \{2, 3\}$$

$$\min \begin{cases} 8 + g(2, \{3\}) \\ 9 + g(3, \{2\}) \end{cases} = \min \begin{cases} 8 + 15 \\ 9 + 18 \end{cases} = \min\{23, 27\} = 23$$

Stage 4) Compute $g(i, S)$ where $|S|=3$, **with three intermediate nodes**

$$i \in (V - \{1\}) \text{ and } i = \{1\},$$

$$S = \{2, 3, 4\}$$

$$j = \{2/3/4\}$$

$$g(i, S) = \min_{j \in S} \{C_{ij} + g(j, S - \{j\})\}$$

$$g(1, \{2, 3, 4\}) = \min \begin{cases} C_{12} + g(2, S - \{2\}) \\ C_{13} + g(3, S - \{3\}) \\ C_{14} + g(4, S - \{4\}) \end{cases} \begin{matrix} \text{when } j = 2 \\ \text{when } j = 3 \\ \text{when } j = 4 \end{matrix}$$

$$\min \begin{cases} 10 + g(2, \{3, 4\}) \\ 15 + g(3, \{2, 4\}) \\ 20 + g(4, \{2, 3\}) \end{cases} = \min \begin{cases} 10 + 25 \\ 15 + 25 \\ 20 + 23 \end{cases} = \min\{35, 40, 43\} = 35$$

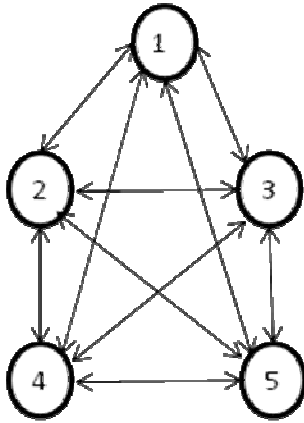
Optimal cost of tour is 35

$$\begin{aligned} g(1, \{2, 3, 4\}) &= \{C_{12} + g(2, \{3, 4\})\} && \text{thus the tour starts from 1 and goes to 2} \\ g(2, \{3, 4\}) &= \{C_{24} + g(4, \{3\})\} && \text{then from 2 to 4} \\ g(4, \{3\}) &= \{C_{43} + g(3, \{\Phi\})\} && \text{then from 4 to 3} \\ g(3, \{\Phi\}) &= C_{31} && \text{and from 3 to 1} \end{aligned}$$

$$C_{12} \quad C_{24} \quad C_{43} \quad C_{31}$$

The sequence of tour is 1 2 4 3 1

Example 2) Travelling Salesperson



	1	2	3	4	5
1	0	2	1	2	1
2	1	0	3	2	5
3	6	2	0	2	1
4	1	1	2	0	2
5	3	1	2	1	0

Compute $g(1, \{2,3,4,5\})$

$V = \{1,2,3,4,5\}$

Stage 1) Compute $g(i,s)$ where $|s| = \emptyset$, $i \in \{v - \{s\}\}$
 $i = \{2,3,4,5\}$ with no intermediate nodes

$$g(2, \emptyset) = C_{21} = 1$$

$$g(3, \emptyset) = C_{31} = 6$$

$$g(4, \emptyset) = C_{41} = 1$$

$$g(5, \emptyset) = C_{51} = 3$$

Stage 2) Compute $g(i,s)$ where $|s| = 1$, $i \in \{v - \{s\}\}$

With one intermediate node

$i = \{2,3,4,5\}$ $s = \{2,3,4,5\}$ & $\langle i, s \rangle$ should be an edge

$$g(i, S) = \min_{j \in S} \{ C_{ij} + g(j, S - \{j\}) \}$$

$$g(2, \{3\}) = C_{23} + g(3, s - \{3\}) = 3 + g(3, \emptyset) = 3 + 6 = 9$$

$$g(2, \{4\}) = C_{24} + g(4, s - \{4\}) = 2 + g(4, \emptyset) = 2 + 1 = 3$$

$$g(2, \{5\}) = C_{25} + g(5, s - \{5\}) = 5 + g(5, \emptyset) = 5 + 3 = 8$$

$$g(3, \{1\}) = C_{31} + g(1, s - \{1\}) = 6 + g(1, \emptyset) = 6 + 0 = 6$$

$$g(3, \{2\}) = C_{32} + g(2, s - \{2\}) = 2 + g(2, \emptyset) = 2 + 1 = 3$$

$$g(3, \{4\}) = C_{34} + g(4, s - \{4\}) = 2 + g(4, \emptyset) = 2 + 1 = 3$$

$$g(3, \{5\}) = C_{35} + g(5, s - \{5\}) = 1 + g(5, \emptyset) = 1 + 3 = 4$$

$$g(4, \{1\}) = C_{41} + g(1, s - \{1\}) = 1 + g(1, \emptyset) = 1 + 0 = 1$$

$$g(4, \{2\}) = C_{42} + g(2, s - \{2\}) = 1 + g(2, \emptyset) = 1 + 1 = 2$$

$$g(4, \{3\}) = C_{43} + g(3, s - \{3\}) = 2 + g(3, \emptyset) = 2 + 6 = 8$$

$$g(4, \{5\}) = C_{45} + g(5, s - \{5\}) = 2 + g(5, \emptyset) = 2 + 3 = 5$$

$$g(5, \{1\}) = C_{51} + g(1, s - \{1\}) = 3 + g(1, \emptyset) = 3 + 0 = 3$$

$$g(5, \{2\}) = C_{52} + g(2, s - \{2\}) = 1 + g(2, \emptyset) = 1 + 1 = 2$$

$$g(5, \{3\}) = C_{53} + g(3, s - \{3\}) = 2 + g(3, \emptyset) = 2 + 6 = 8$$

$$g(5, \{4\}) = C_{54} + g(4, s - \{4\}) = 1 + g(4, \emptyset) = 1 + 1 = 2$$

Stage 3) Compute $g(i,s)$ where $|s|=2$, i.e $i \in \{v - \{s\}\}$

With two intermediate nodes

$$i = \{2, 3, 4, 5\} \quad s = \{2, 3, 4, 5\}$$

$$g(i, S) = \min_{j \in S} \{C_{ij} + g(j, S - \{j\})\}$$

$$g(2, \{3, 4\}) = \min \left\{ \begin{array}{l} C_{23} + g(3, S - \{3\}) \\ C_{24} + g(4, S - \{4\}) \end{array} \right\}$$

$$\min \left\{ \begin{array}{l} 3 + g(3, \{4\}) \\ 2 + g(4, \{3\}) \end{array} \right\} = \min \left\{ \begin{array}{l} 3 + 3 \\ 2 + 8 \end{array} \right\} = \min \left\{ \begin{array}{l} 6 \\ 10 \end{array} \right\} = 6$$

$$g(2, \{4, 5\}) = \min \left\{ \begin{array}{l} C_{24} + g(4, S - \{4\}) \\ C_{25} + g(5, S - \{5\}) \end{array} \right\}$$

$$\min \left\{ \begin{array}{l} 2 + g(4, \{5\}) \\ 5 + g(5, \{4\}) \end{array} \right\} = \min \left\{ \begin{array}{l} 2 + 5 \\ 5 + 2 \end{array} \right\} = \min \left\{ \begin{array}{l} 7 \\ 7 \end{array} \right\} = 7$$

$$g(3, \{2, 4\}) = \min \left\{ \begin{array}{l} C_{32} + g(2, S - \{2\}) \\ C_{34} + g(4, S - \{4\}) \end{array} \right\}$$

$$\min \left\{ \begin{array}{l} 2 + g(2, \{4\}) \\ 2 + g(4, \{2\}) \end{array} \right\} = \min \left\{ \begin{array}{l} 2 + 3 \\ 2 + 2 \end{array} \right\} = \min \left\{ \begin{array}{l} 5 \\ 4 \end{array} \right\} = 4$$

$$g(3, \{2, 5\}) = \min \left\{ \begin{array}{l} C_{32} + g(2, S - \{2\}) \\ C_{35} + g(5, S - \{5\}) \end{array} \right\}$$

$$\min \left\{ \begin{array}{l} 2 + g(2, \{5\}) \\ 1 + g(5, \{2\}) \end{array} \right\} = \min \left\{ \begin{array}{l} 2 + 8 \\ 1 + 2 \end{array} \right\} = \min \left\{ \begin{array}{l} 10 \\ 3 \end{array} \right\} = 3$$

$$g(4, \{2, 3\}) = \min \left\{ \begin{array}{l} C_{42} + g(2, S - \{2\}) \\ C_{43} + g(3, S - \{3\}) \end{array} \right\}$$

$$\min \left\{ \begin{array}{l} 1 + g(2, \{3\}) \\ 2 + g(3, \{2\}) \end{array} \right\} = \min \left\{ \begin{array}{l} 1 + 9 \\ 2 + 3 \end{array} \right\} = \min \left\{ \begin{array}{l} 10 \\ 5 \end{array} \right\} = 5$$

$$g(4, \{2, 5\}) = \min \left\{ \begin{array}{l} C_{42} + g(2, S - \{2\}) \\ C_{45} + g(5, S - \{5\}) \end{array} \right\}$$

$$\min \left\{ \begin{array}{l} 1 + g(2, \{5\}) \\ 2 + g(5, \{2\}) \end{array} \right\} = \min \left\{ \begin{array}{l} 1 + 5 \\ 2 + 2 \end{array} \right\} = \min \left\{ \begin{array}{l} 6 \\ 4 \end{array} \right\} = 4$$

$$g(5, \{2, 3\}) = \min \left\{ \begin{array}{l} C_{52} + g(2, S - \{2\}) \\ C_{53} + g(3, S - \{3\}) \end{array} \right\}$$

$$\min \left\{ \begin{array}{l} 1 + g(2, \{3\}) \\ 2 + g(3, \{2\}) \end{array} \right\} = \min \left\{ \begin{array}{l} 1 + 9 \\ 2 + 3 \end{array} \right\} = \min \left\{ \begin{array}{l} 10 \\ 5 \end{array} \right\} = 5$$

$$g(5, \{2, 4\}) = \min \left\{ \begin{array}{l} C_{52} + g(2, S - \{2\}) \\ C_{54} + g(4, S - \{4\}) \end{array} \right\}$$

$$\min \left\{ \begin{array}{l} 1 + g(2, \{4\}) \\ 1 + g(4, \{2\}) \end{array} \right\} = \min \left\{ \begin{array}{l} 1 + 3 \\ 1 + 2 \end{array} \right\} = \min \left\{ \begin{array}{l} 4 \\ 3 \end{array} \right\} = 3$$

$$g(2,\{3,5\}) = \min \begin{Bmatrix} C_{23} + g(3,S-\{3\}) \\ C_{25} + g(5,S-\{5\}) \end{Bmatrix}$$

$$\min \begin{Bmatrix} 3 + g(3,\{5\}) \\ 5 + g(5,\{3\}) \end{Bmatrix} = \min \begin{Bmatrix} 3 + 4 \\ 5 + 8 \end{Bmatrix} = \min \begin{Bmatrix} 7 \\ 13 \end{Bmatrix} = 7$$

$$g(3,\{4,5\}) = \min \begin{Bmatrix} C_{34} + g(4,S-\{4\}) \\ C_{35} + g(5,S-\{5\}) \end{Bmatrix}$$

$$\min \begin{Bmatrix} 2 + g(4,\{5\}) \\ 1 + g(5,\{4\}) \end{Bmatrix} = \min \begin{Bmatrix} 2 + 5 \\ 1 + 2 \end{Bmatrix} = \min \begin{Bmatrix} 7 \\ 3 \end{Bmatrix} = 3$$

$$g(4,\{3,5\}) = \min \begin{Bmatrix} C_{43} + g(3,S-\{3\}) \\ C_{45} + g(5,S-\{5\}) \end{Bmatrix}$$

$$\min \begin{Bmatrix} 2 + g(3,\{5\}) \\ 2 + g(5,\{3\}) \end{Bmatrix} = \min \begin{Bmatrix} 2 + 4 \\ 2 + 8 \end{Bmatrix} = \min \begin{Bmatrix} 6 \\ 10 \end{Bmatrix} = 6$$

$$g(5,\{3,4\}) = \min \begin{Bmatrix} C_{53} + g(3,S-\{3\}) \\ C_{54} + g(4,S-\{4\}) \end{Bmatrix}$$

$$\min \begin{Bmatrix} 2 + g(3,\{4\}) \\ 1 + g(4,\{3\}) \end{Bmatrix} = \min \begin{Bmatrix} 2 + 3 \\ 1 + 8 \end{Bmatrix} = \min \begin{Bmatrix} 5 \\ 9 \end{Bmatrix} = 5$$

Stage 4) Compute $g(i,s)$ where $|s|=3$, i.e. $i \in \{v - \{s\}\}$

With three intermediate nodes

$$i = \{2,3,4,5\} \quad s = \{2,3,4,5\}$$

$$g(i, S) = \min_{j \in S} \{ C_{ij} + g(j, S - \{j\}) \}$$

$$g(2,\{3,4,5\}) = \min \begin{Bmatrix} C_{23} + g(3,S-\{3\}) \\ C_{24} + g(4,S-\{4\}) \\ C_{25} + g(5,S-\{5\}) \end{Bmatrix}$$

$$= \min \begin{Bmatrix} 3 + g(3,\{4,5\}) \\ 2 + g(4,\{3,5\}) \\ 5 + g(5,\{3,4\}) \end{Bmatrix} = \min \begin{Bmatrix} 3 + 3 \\ 2 + 6 \\ 5 + 5 \end{Bmatrix} = \min \begin{Bmatrix} 6 \\ 8 \\ 10 \end{Bmatrix} = 6$$

$$g(3,\{2,4,5\}) = \min \begin{Bmatrix} C_{32} + g(2,S-\{2\}) \\ C_{34} + g(4,S-\{4\}) \\ C_{35} + g(5,S-\{5\}) \end{Bmatrix}$$

$$= \min \begin{Bmatrix} 2 + g(2,\{4,5\}) \\ 2 + g(4,\{2,5\}) \\ 1 + g(5,\{2,4\}) \end{Bmatrix} = \min \begin{Bmatrix} 2 + 7 \\ 2 + 4 \\ 1 + 3 \end{Bmatrix} = \min \begin{Bmatrix} 9 \\ 6 \\ 4 \end{Bmatrix} = 4$$

$$g(4,\{2,3,5\}) = \min \begin{Bmatrix} C_{42} + g(2,S-\{2\}) \\ C_{43} + g(3,S-\{3\}) \\ C_{45} + g(5,S-\{5\}) \end{Bmatrix}$$

$$= \min \begin{Bmatrix} 1 + g(2,\{3,5\}) \\ 2 + g(3,\{2,5\}) \\ 2 + g(5,\{2,3\}) \end{Bmatrix} = \min \begin{Bmatrix} 1 + 7 \\ 2 + 3 \\ 2 + 5 \end{Bmatrix} = \min \begin{Bmatrix} 8 \\ 5 \\ 7 \end{Bmatrix} = 5$$

$$\begin{aligned}
g(5, \{2,3,4\}) &= \min \left\{ \begin{array}{l} C_{52} + g(2, S - \{2\}) \\ C_{53} + g(3, S - \{3\}) \\ C_{54} + g(4, S - \{4\}) \end{array} \right\} \\
&= \min \left\{ \begin{array}{l} 1 + g(2, \{3,4\}) \\ 2 + g(3, \{2,4\}) \\ 1 + g(4, \{2,3\}) \end{array} \right\} = \min \left\{ \begin{array}{l} 1 + 6 \\ 2 + 4 \\ 1 + 5 \end{array} \right\} = \min \left\{ \begin{array}{l} 7 \\ 6 \\ 6 \end{array} \right\} = 6
\end{aligned}$$

Stage 5) Compute $g(i,s)$ where $|s|=4$, i.e. $i \in \{v - \{1\}\}$

With 4 intermediate nodes

$$i = \{1\} \quad s = \{2,3,4,5\}$$

$$g(i, S) = \min_{j \in S} \{ C_{ij} + g(j, S - \{j\}) \}$$

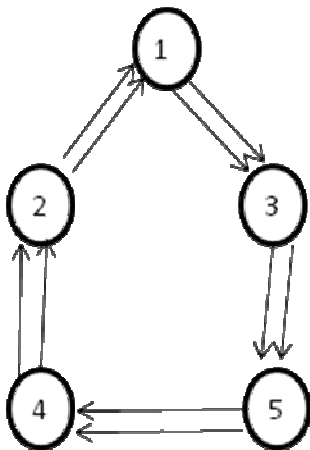
$$g(1, \{2,3,4,5\}) = \min \left\{ \begin{array}{l} C_{12} + g(2, S - \{2\}) \\ C_{13} + g(3, S - \{3\}) \\ C_{14} + g(4, S - \{4\}) \\ C_{15} + g(5, S - \{5\}) \end{array} \right\}$$

$$\begin{aligned}
&= \min \left\{ \begin{array}{l} 2 + g(2, \{3,4,5\}) \\ 1 + g(3, \{2,4,5\}) \\ 2 + g(4, \{2,3,5\}) \\ 1 + g(5, \{2,3,4\}) \end{array} \right\} = \min \left\{ \begin{array}{l} 2 + 6 \\ 1 + 4 \\ 2 + 5 \\ 1 + 6 \end{array} \right\} = \min \left\{ \begin{array}{l} 8 \\ 5 \\ 7 \\ 7 \end{array} \right\} = 5
\end{aligned}$$

Optimal Cost tour is 5

$$\begin{array}{lll}
g(1, \{2,3,4,5\}) &= C_{13} + g(3, \{2,4,5\}) & \text{thus tour starts from 1 and goes to 3} \\
g(3, \{2,4,5\}) &= C_{35} + g(5, \{2,4\}) & \text{then from 3 to 5} \\
g(5, \{2,4\}) &= C_{52} + g(2, \{4\}) & \text{then from 5 to 2} \\
g(2, \{4\}) &= C_{24} + g(4, \{\emptyset\}) & \text{then from 2 to 4} \\
g(4, \emptyset) &= C_{41} & \text{then from 4 to 1}
\end{array}$$

Optimal tour is 1-3-5-4-2-1



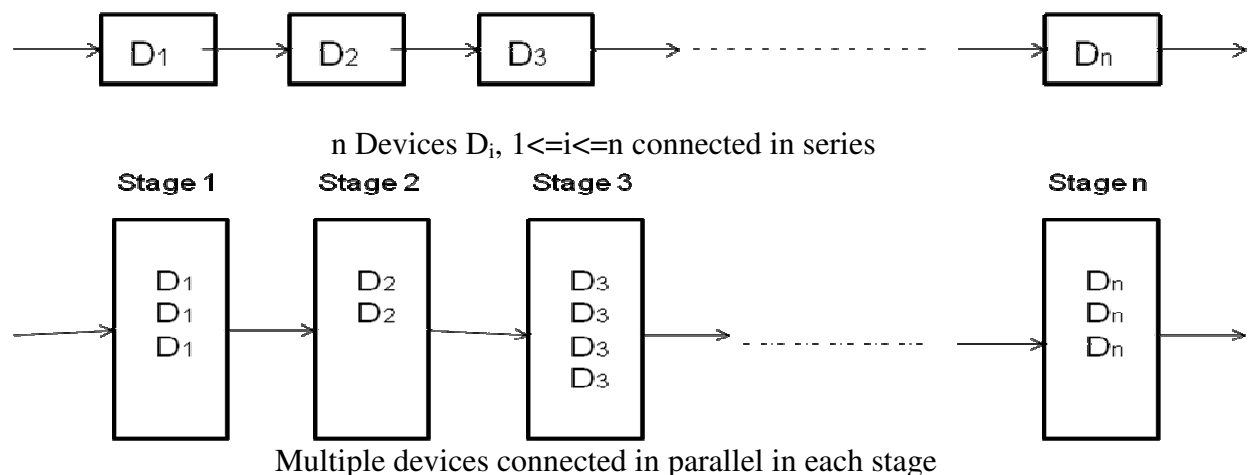
6)RELIABILITY DESIGN:

In this section we look at an example of how to use dynamic programming to solve a problem with a multiplicative optimization function. The problem is to design a system that is composed of several devices connected in series. Let r_i be the reliability of device D_i . Then the reliability of the entire system is $\prod r_i$. Even if the individual devices are very reliable, the reliability of the system may not be very good.

For example if $n=10$ and $r_i=0.99$, $1 \leq i \leq 10$ then $\prod r_i = 0.904$. Hence it is desirable to duplicate devices. Multiple copies of same device type connected in parallel through the use of switching circuits. The switching circuits determine which devices in any given group are functioning properly. They then make use of one such device at each stage.

If stage i contains m_i copies of device d_i , then the probability that all m_i have a malfunction is $(1 - r_i)^{m_i}$. Hence the reliability of stage i becomes $1 - (1 - r_i)^{m_i}$. Thus if $r_i = 0.80$ and $m_i = 2$, the stage reliability becomes 0.96.

$$\begin{aligned} \text{Reliability} &= 1 - (1 - 0.80)^2 \\ &= 1 - (0.2)^2 \\ &= 1 - 0.04 \\ &= 0.96 \end{aligned} \quad \text{The reliability of this stage increased with duplication.}$$



Our problem is to use device duplication to maximize reliability. The maximization is to be carried out under a cost constraint. Let C_i be the cost of system being designed we wish to solve the following maximization problem.

$$\begin{aligned} &\text{Maximize } \prod \phi_i(m_i) \quad (\text{reliability function}) \\ &\text{Subject to } \sum_{1 \leq i \leq n} c_i m_i \leq C \quad (\text{total allowable cost}) \quad m_i \geq c_i \quad 1 \leq i \leq n \end{aligned}$$

We may assume that each $c_i > 0$

Each m_i must be in the range $1 \leq m_i \leq U_i$ can be calculated as follows

Upper bound is the maximum number of devices that can be kept in one stage.

$$U_i = \left\lfloor \left(c + c_i - \sum_{j=1}^n c_j \right) / c_i \right\rfloor$$

Clearly $F_0(x) = 1 \quad \forall x \quad 0 \leq x \leq c$

It may be solved using an approach similar to that used for knapsack problem.

Example : we are to design a 3 stage system with device types d1, D2, d3. The costs are \$30, \$15 and \$20 respectively. The cost of the system is to be no more than \$105.

The reliability of each device type is 0.9, 0.8 and 0.5 respectively.
We assume that if stage i has m_i devices of type i in parallel then

$$\phi_i(m_i) = 1 - (1 - r_i)^{m_i}$$

In terms of the notation used earlier,
 $c_1=30, c_2=15, c_3=20, c=105$
 $r_1=0.9, r_2=0.8, r_3=0.5$

$$U_i = \left\lfloor \left(c + c_i - \sum_{j=1}^n c_j \right) / c_i \right\rfloor \quad \text{upper bound}$$

Upper bound at Stage 1

$$U_1 = \lfloor (105 + 30 - (30 + 15 + 20)) / 30 \rfloor$$

$$U_1 = \lfloor (135 - 65) / 30 \rfloor$$

$$U_1 = \lfloor 2.333 \rfloor$$

$$U_1 = 2$$

Upper bound at Stage 2

$$U_2 = \lfloor (105 + 15 - (30 + 15 + 20)) / 15 \rfloor$$

$$U_2 = \lfloor (120 - 65) / 15 \rfloor$$

$$U_2 = \lfloor 3.666 \rfloor$$

$$U_2 = 3$$

Upper bound at Stage 3

$$U_3 = \lfloor (105 + 20 - (30 + 15 + 20)) / 20 \rfloor$$

$$U_3 = \lfloor (125 - 65) / 20 \rfloor$$

$$U_3 = \lfloor 3 \rfloor$$

$$U_3 = 3$$

$$U_1 = 2$$

$$U_2 = 3$$

$$U_3 = 3$$

Reliability function for i^{th} stage

$\phi_i(m_i) = 1 - (1 - r_i)^{m_i}$ is a reliability function. Each pair in this is represented by (r,x) where r is reliability and x is cost.

Using S_j^i to represent all tuples that are obtainable from S^{i-1} by choosing $m_i = j$.

Beginning with $S^0 = \{(1,0)\}$ initial assumption
 {reliability, cost}

As there are no devices no chance for failure. So reliability is 1.

As there are no devices cost of devices is 0.

Since $1 \leq m_i \leq U_i$

$$1 \leq m_1 \leq 2 \quad m_1=1 \text{ or } 2, \text{ it means } j=1 \text{ or } 2.$$

So, by using S_j^i calculate S_1^1 here $i=1, j=1$ i.e. $m_1=1$

The reliability function for ith stage is

$$\phi_i(m_i) = 1 - (1 - r_i)^{m_i}$$

$$\phi_1(m_1) = 1 - (1 - r_1)^1 \quad \text{reliability function for 1st stage}$$

$$= 1 - (1 - 0.9)$$

$$= 1 - 0.1$$

$$= 0.9$$

$$(\text{Reliability, cost}) = (0.9, 30)$$

$$S^0 = \{(1, 0)\} \quad \text{no devices}$$

In each tuple (r,x) reliability is multiplied with previous reliability and cost is added to previous cost.

$$S_1^1 = \{(1 \times 0.9, 0 + 30)\}$$

$$S_1^1 = \{(0.9, 30)\} \quad \text{stage}=1 \quad \text{number of devices}=1$$

Here reliability 0.9 is multiplied with 1 and cost c1=30 is added to 0.

Now S_2^1 is calculated as follows. i.e. Stage=1, number of devices =2

$$i=1 \quad j=2 \quad m_1=2$$

$$\phi_i(m_i) = 1 - (1 - r_i)^{m_i}$$

$$\phi_1(m_1) = 1 - (1 - r_1)^2$$

$$= 1 - (1 - 0.9)^2$$

$$= 1 - 0.01$$

$$= 0.99$$

$$(\text{Reliability, cost}) = (0.99, 60)$$

$$S_2^1 = \{(1 \times 0.99, 0 + 60)\}$$

$$S_2^1 = \{(0.99, 60)\}$$

$$S^1 \text{ can be obtained by merging the sets } S^1 = S_1^1 \cup S_2^1$$

$$S^1 = \{(0.9, 30), (0.99, 60)\}$$

We cannot calculate S_3^1 since upper bound is 2 i.e. $U_1=2$.

Similarly S_1^2, S_2^2, S_3^2 can be calculated from S^1 .

Stage 2 one device

$$S_1^2 \text{ here } i=2, j=1 \text{ i.e. } m_2=1$$

$$\phi_2(m_2) = 1 - (1 - r_2)^{m_2}$$

$$\phi_2(m_2) = 1 - (1 - 0.8)^1$$

$$= 1 - 0.2$$

$$= 0.8$$

(Reliability,cost)=(0.8,15)

$$S_1^2 = \{(0.9 \times 0.8, 30 + 15), (0.99 \times 0.8, 60 + 15)\}.$$

$$S_1^2 = \{(0.72, 45), (0.792, 75)\}$$

Stage 2 two devices

$$S_2^2 \text{ here } i=2, j=2 \text{ i.e. } m_2 = 2$$

$$\phi_2(m_2) = 1 - (1 - r_2)^{m_2}$$

$$\begin{aligned}\phi_2(m_2) &= 1 - (1 - 0.8)^2 \\ &= 1 - 0.2 * 0.2 \\ &= 0.96\end{aligned}$$

(reliability,cost)=(0.96,30)

Since $j=2$, cost of 2 devices $2C_2 = 2 * 15 = 30$ is added to S^1 pairs.

$$S_2^2 = \{(0.9 \times 0.96, 30 + 30), (0.99 \times 0.96, 60 + 30)\}.$$

$$S_2^2 = \{(0.864, 60), (0.9504, 90)\}$$

Stage 2 three devices

$$S_3^2 \text{ here } i=2, j=3 \text{ i.e. } m_2 = 3$$

$$\phi_2(m_2) = 1 - (1 - r_2)^{m_2}$$

$$\begin{aligned}\phi_2(m_2) &= 1 - (1 - 0.8)^3 \\ &= 1 - 0.2^3 \\ &= 0.992\end{aligned}$$

(reliability,cost)=(0.992,45)

Since $j=3$, cost of 3 devices $3C_2 = 3 * 15 = 45$ is added to S^1 pairs.

$$S_3^2 = \{(0.9 \times 0.992, 30 + 45), (0.99 \times 0.992, 60 + 45)\}.$$

$$S_3^2 = \{(0.8928, 75), (0.98208, 105)\}$$

S^2 can be obtained by merging the sets $S^2 = S_1^2 \cup S_2^2 \cup S_3^2$

$$S^2 = \{(0.72, 45), (0.792, 75), (0.864, 60), (0.9504, 90), (0.8928, 75), (0.98208, 105)\}$$

Now by applying purge rule : by observing the successive tuples (0.792,75),(0.864,60) the second one dominates first one as the second one has more reliability with less cost. So we purge i.e. eliminate the tuple (0.792,75).

$$S^2 = \{(0.72, 45), (0.864, 60), (0.9504, 90), (0.8928, 75), (0.98208, 105)\}$$

Now observe that the cost must be in increasing order. $45 < 60 < 90 < 75 < 105$ false
i.e. $90 < 75$ fails so pair (0.9504,90) is not in appropriate order so remove (0.9504,90).

$$S^2 = \{(0.72, 45), (0.864, 60), (0.8928, 75), (0.98208, 105)\}$$

Similarly S_1^3, S_2^3, S_3^3 can be calculated from S^2 .

Stage 3 one device

S_1^3 here $i=3, j=1$ i.e. $m_3 = 1$

$$\phi_3(m_3) = 1 - (1 - r_3)^{m_3}$$

$$\phi_3(m_3) = 1 - (1 - 0.5)^1$$

$$= 1 - 0.5$$

$$= 0.5$$

Cost of the device at stage 3 = 20. So (0.5, 20) is added to S^2

$$S_1^3 = \{(0.72 \times 0.5, 45 + 20), (0.864 \times 0.5, 60 + 20), (0.8928 \times 0.5, 75 + 20), (0.98208 \times 0.5, 105 + 20)\}$$

Last tuple is removed from S_1^3 since cost exceeds budget 105.

$$S_1^3 = \{(0.72 \times 0.5, 45 + 20), (0.864 \times 0.5, 60 + 20), (0.8928 \times 0.5, 75 + 20)\}$$

$$S_1^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95)\}$$

Stage 3 two devices

S_2^3 here $i=3, j=2$ i.e. $m_3 = 2$

$$\phi_3(m_3) = 1 - (1 - r_3)^{m_3}$$

$$\phi_3(m_3) = 1 - (1 - 0.5)^2$$

$$= 1 - 0.25 \times 0.25$$

$$= 0.75$$

reliability

Since $j=2$, cost of 2 devices $2C_2 = 2 \times 20 = 40$ so the pair (0.75, 40) is added to S^2 pairs.

$$S_2^3 = \{(0.72 \times 0.75, 45 + 40), (0.864 \times 0.75, 60 + 40)\}$$

$$S_2^3 = \{(0.54, 85), (0.648, 100)\}$$

Stage 3 three devices

S_3^3 here $i=3, j=3$ i.e. $m_3 = 3$

$$\phi_3(m_3) = 1 - (1 - r_3)^{m_3}$$

$$\phi_3(m_3) = 1 - (1 - 0.5)^3$$

$$= 1 - 0.5^3$$

$$= 0.875$$

reliability

Since $j=3$, cost of 3 devices $3C_2 = 3 \times 20 = 60$ the pair (0.875, 60) is added to S^2 pairs.

$$S_3^3 = \{(0.72 \times 0.875, 45 + 60)\}$$

$$S_3^3 = \{(0.63, 105)\}$$

In all other pairs the cost exceeds 105 so not included.

S^3 can be obtained by merging the sets $S^3 = S_1^3 \cup S_2^3 \cup S_3^3$

$$S^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95), (0.54, 85), (0.648, 100), (0.63, 105)\}$$

Now by applying purge and dominance rule : by observing the successive tuples (0.4464,95),(0.54,85) the second one dominates first one as the second one has more reliability with less cost. So we purge i.e. eliminate the tuple (0.4464,95) and by observing the successive tuples (0.648,100),(0.63,105) the first one dominates second one. As the first one has more reliability with less cost we purge i.e. eliminate the tuple (0.63,105)

$$S^3 = \{(0.36, 65), (0.432, 80), (0.54, 85), (0.648, 100)\}$$

The best design has reliability of 0.648 and a cost of 100.

The pair (0.648,100) present in S_2^3 i.e. $i=3$ and $j=2$ two devices at stage 3
so $m_3=2$ cost = 40

The pair (0.648,100) obtained from (0.864, 60) which is present in S_2^2 i.e. $i=2, j=2$ so $m_2=2$
i.e two devices at stage two. So cost will be $15*2=30$ at stage two.

The pair (0.864,60) obtained from (0.9,30) which is in S_1^1 i.e. $i=1, j=1$ so $m_1=1$
One device at stage 1 i.e cost =30

$$\therefore m_1=1, m_2=2, m_3=2$$

An alternate method for finding the solution for the above problem:

The pair of tuple with the maximum reliability is in S^3 represents the value of the solution. Therefore the reliability of the system is 0.648 that has a cost of 100. To get the values of M_1 , M_2 and M_3 we will trace back.

The pair $(0.648, 100) \in S_2^3$ $i=3, j=2, m_3=2$

As there are 2 copies of the device at stage 3 subtracting the cost of 2 copies of the device D3 from the total cost 100 i.e
 $= 100 - 2*20$
 $= 100-40$
 $= 60.$

Search (x,60) belongs to S_1^2 or S_2^2 or S_3^2

$$(0.864, 60) \in S_2^2 \quad i=2, j=2, m_2=2$$

(0.864,60) belongs to S_2^2 so $m_2=2$ as there are 2 devices at stage 2 the cost of 2 copies of device type D2 is subtracted from total cost 60
 $=60-2*15$
 $=60-30$
 $=30$

Now Search (x,30) belongs to S_1^1 or S_2^1

$$(0.9, 30) \in S_1^1 \quad i=1, j=1, m_1=1$$

(0.9,30) belongs to S_1^1 i.e $m_1=1$

$$\therefore m_1=1, m_2=2, m_3=2$$

7) All pairs shortest path problem

Let $G=(V,E)$ be a directed graph consisting of n vertices and each edge is associated with a weight. The problem of finding the shortest path between all pairs of vertices in a graph is called all pairs shortest path problem. This problem can be solved by using Dynamic programming Technique.

The all pair shortest path problem is to determine a matrix A such that $A(i,j)$ is the length of a shortest path from vertex i to j . Assume that this path contains no cycles. If k is an intermediate vertex on this path, then the sub paths from i to k and from k to j are the shortest paths from i to k and from k to j respectively.

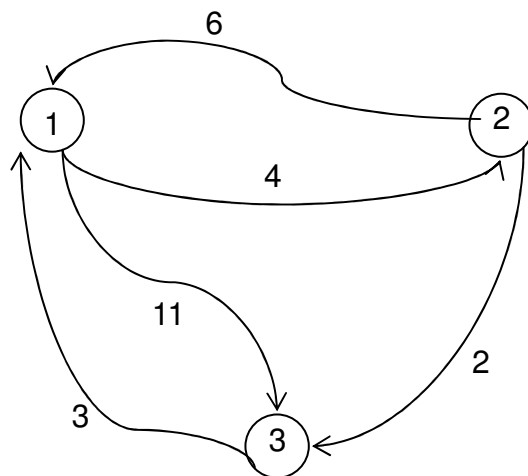
Otherwise the path from i to j is not shortest path. If k is intermediate vertex with highest index then the path i to k is the shortest path going through no vertex with index greater than $k-1$. similarly the path k to j is shortest path going through no vertex with index greater than $k-1$.

The shortest path can be computed using following recursive method.

$$A^k(i, j) = W(i, j) \quad \text{if } k = 0$$

$$A^k(i, j) = \min\{A^{k-1}(i, j), A^{k-1}(i, k) + A^{k-1}(k, j)\} \quad \text{if } k \geq 1$$

Example : Compute all pairs shortest path for the following graph



Graph G

$$\text{cost Adjacency Matrix } A^0(i, j) = W(i, j) = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix}$$

Step 1

For k=1 i.e. going from i to j through intermediate vertex 1

When i=1 j=1/2/3

$$A^1(1,1)=\min\{A^0(1,1), A^0(1,1)+A^0(1,1)\}=\min\{0,0+0\}=0$$

$$A^1(1,2)=\min\{A^0(1,2), A^0(1,1)+A^0(1,2)\}=\min\{4,0+4\}=4$$

$$A^1(1,3)=\min\{A^0(1,3), A^0(1,1)+A^0(1,3)\}=\min\{11,0+11\}=11$$

When i=2 j=1/2/3

$$A^1(2,1)=\min\{A^0(2,1), A^0(2,1)+A^0(1,1)\}=\min\{6,6+0\}=6$$

$$A^1(2,2)=\min\{A^0(2,2), A^0(2,1)+A^0(1,2)\}=\min\{0,6+4\}=0$$

$$A^1(2,3)=\min\{A^0(2,3), A^0(2,1)+A^0(1,3)\}=\min\{2,6+11\}=2$$

When i=3 j=1/2/3

$$A^1(3,1)=\min\{A^0(3,1), A^0(3,1)+A^0(1,1)\}=\min\{3,3+0\}=3$$

$$A^1(3,2)=\min\{A^0(3,2), A^0(3,1)+A^0(1,2)\}=\min\{\infty,3+4\}=7$$

$$A^1(3,3)=\min\{A^0(3,3), A^0(3,1)+A^0(1,3)\}=\min\{0,3+11\}=0$$

$$\text{cost Adjacency Matrix } A^1(i, j) = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

Step 2

For k=2 i.e. going from i to j through intermediate vertex 2

When i=1 k=2 j=1/2/3

$$A^k(i, j)=\min\{A^{k-1}(i, j), A^{k-1}(i, k)+A^{k-1}(k, j)\}$$

$$A^2(1,1)=\min\{A^1(1,1), A^1(1,2)+A^1(2,1)\}=\min\{0,4+6\}=0$$

$$A^2(1,2)=\min\{A^1(1,2), A^1(1,2)+A^1(2,2)\}=\min\{4,4+0\}=4$$

$$A^2(1,3)=\min\{A^1(1,3), A^1(1,2)+A^1(2,3)\}=\min\{11,4+2\}=6$$

When i=2 j=1/2/3

$$A^2(2,1)=\min\{A^1(2,1), A^1(2,2)+A^1(2,1)\}=\min\{6,0+6\}=6$$

$$A^2(2,2)=\min\{A^1(2,2), A^1(2,2)+A^1(2,2)\}=\min\{0,0+0\}=0$$

$$A^2(2,3)=\min\{A^1(2,3), A^1(2,2)+A^1(2,3)\}=\min\{2,0+2\}=2$$

When i=3 j=1/2/3

$$A^2(3,1)=\min\{A^1(3,1), A^1(3,2)+A^1(2,1)\}=\min\{3,7+6\}=3$$

$$A^2(3,2)=\min\{A^1(3,2), A^1(3,2)+A^1(2,2)\}=\min\{7,7+0\}=7$$

$$A^2(3,3)=\min\{A^1(3,3), A^1(3,2)+A^1(2,3)\}=\min\{0,0+2\}=0$$

$$\text{cost Adjacency Matrix } A^2(i, j) = \begin{bmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

Step 3

For k=3 i.e. going from i to j through intermediate vertex 3

When i=1 k=3 j=1/2/3

$$A^k(i, j) = \min\{A^{k-1}(i, j), A^{k-1}(i, k) + A^{k-1}(k, j)\}$$

$$A^3(1, 1) = \min\{A^2(1, 1), A^2(1, 3) + A^2(3, 1)\} = \min\{0, 6 + 3\} = 0$$

$$A^3(1, 2) = \min\{A^2(1, 2), A^2(1, 3) + A^2(3, 2)\} = \min\{4, 6 + 7\} = 4$$

$$A^3(1, 3) = \min\{A^2(1, 3), A^2(1, 3) + A^2(3, 3)\} = \min\{6, 6 + 0\} = 6$$

When i=2 j=1/2/3

$$A^3(2, 1) = \min\{A^2(2, 1), A^2(2, 3) + A^2(3, 1)\} = \min\{6, 2 + 3\} = 5$$

$$A^3(2, 2) = \min\{A^2(2, 2), A^2(2, 3) + A^2(3, 2)\} = \min\{0, 2 + 7\} = 0$$

$$A^3(2, 3) = \min\{A^2(2, 3), A^2(2, 3) + A^2(3, 3)\} = \min\{2, 2 + 0\} = 2$$

When i=3 j=1/2/3

$$A^3(3, 1) = \min\{A^2(3, 1), A^2(3, 3) + A^2(3, 1)\} = \min\{3, 0 + 3\} = 3$$

$$A^3(3, 2) = \min\{A^2(3, 2), A^2(3, 3) + A^2(3, 2)\} = \min\{7, 0 + 7\} = 7$$

$$A^3(3, 3) = \min\{A^2(3, 3), A^2(3, 3) + A^2(3, 3)\} = \min\{0, 0 + 0\} = 0$$

$$\text{cost Adjacency Matrix } A^3(i, j) = \begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

This matrix gives the all pairs shortest path solution

Algorithm all_pairs_shortest_path(W,A,n)

// W is weighted array matrix, n is the number of vertices,

// A is the cost of shortest path from vertex i to j.

```
{
  for i:= 1 to n do
    for j:= 1 to n do
      A[i,j] := W[i,j]

  for k:=1 to n do
    for i:= 1 to n do
      for j:= 1 to n do
        A[i,j] := min(A[i,j], A[i,k] + A[k,j])
}
```

The Time Complexity for this method is $O(n^3)$

Important Questions

- 1) Devise an algorithm to find the optimal order of multiplying n matrices using dynamic programming technique.
- 2) Write an algorithm of matrix chain multiplication.
- 3) Write a pseudo code of the dynamic programming algorithm for solving optimal binary search tree.
- 4) Using algorithm of OBST, compute $w(i,j)$, $r(i,j)$ and $c(i,j)$, $0 \leq i \leq j \leq 4$ for identifier set $(a_1, a_2, a_3, a_4) = (\text{cout}, \text{float}, \text{int}, \text{while})$ with $p_1=1/20$, $p_2=1/5$, $p_3=1/10$, $p_4=1/20$, $q_0=1/5$, $q_1=1/10$, $q_2=1/5$, $q_3=1/20$, $q_4=1/20$ using $r(i,j)$ construct the optimal binary search tree.
- 5) Write an algorithm of all pair shortest path problem
- 6) Discuss the dynamic programming solutions for the traveling sales person problem
- 7) Discuss the dynamic programming solutions for the problems of reliability design.
- 8) Design a system with 3 components D1, D2, D3 each costs Rs.30, Rs.15 Rs.30 respectively. The cost of the system is to be no more than Rs.105. The reliability of each device type is 0.9, 0.8 and 0.5 respectively.
- 9) Discuss 0/1 knapsack problem. Consider $n=3$, $(w_1, w_2, w_3)=(2, 3, 4)$ $(p_1, p_2, p_3)=(1, 2, 5)$ and $m=6$